

HW4: Palmer Penguins — Summary Statistics & Plots

Setup

- Use only `pandas`, `matplotlib`, and `seaborn`.
- Do your work in the cells provided; you may add extra cells.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# (Optional) nicer display for DataFrames
pd.set_option("display.max_columns", 50)
pd.set_option("display.precision", 3)
```

```
In [2]: ### Load the dataset
penguins = pd.read_csv("penguins.csv")

# Standardize column names (snake_case)
penguins.columns = [c.strip().lower().replace(' ', '_') for c in penguins.columns]

penguins.head()
```

```
Out[2]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	male	2007
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	female	2007
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	female	2007
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN	2007
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	female	2007

Part 1) Data Vitals

Answer the following questions.

Your answer to each question must include some code that produces the result. You can, but you do not have to use a complete sentence to answer each question. Add cells as needed.

- a. What is the shape of the data?
- b. What are the Column names and data types of each column?
- c. How many values in each column are missing?
- d. How many penguins are there of each species? How many male and female penguins are there?

```
In [3]: penguins.shape
```

```
Out[3]: (344, 8)
```

The data has 344 rows and 8 columns.

```
In [4]: penguins.dtypes
```

```
Out[4]: species          str
        island          str
        bill_length_mm  float64
        bill_depth_mm   float64
        flipper_length_mm float64
        body_mass_g      float64
        sex             str
        year            int64
        dtype: object
```

The names of the columns are `species`, `island`, `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`, `sex`, and `year`. The types of each column in the same order is string, string, float, float, float, float, sting, and integer.

```
In [5]: penguins.isnull().sum()
```

```
Out[5]: species          0
        island          0
        bill_length_mm    2
        bill_depth_mm     2
        flipper_length_mm  2
        body_mass_g       2
        sex              11
        year              0
        dtype: int64
```

The columns `species`, `island`, and `year` do not have missing values. The columns `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, and `body_mass_g` each contain 2 missing values. The `sex` column has 11 missing values.

```
In [6]: penguins.groupby("species")["species"].count()
```

```
Out[6]: species
        Adelie      152
        Chinstrap   68
        Gentoo     124
        Name: species, dtype: int64
```

The Adelie species contains 152 penguins, the Chinstrap contains 68 penguins, and Gentoo contains 124 penguins.

```
In [7]: penguins.groupby("sex")["sex"].count()
```

```
Out[7]: sex
        female    165
        male      168
        Name: sex, dtype: int64
```

There are 165 female penguins and 168 male penguins.

Part 2) Handle missing data

- a. Create a copy `penguins_clean` that **drops** rows with missing values in `['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']` .
- b. Report the new shape of `penguins_clean` .

```
In [8]: penguins_clean = penguins.copy().dropna()
penguins_clean.isnull().sum()
```

```
Out[8]: species          0
island              0
bill_length_mm      0
bill_depth_mm       0
flipper_length_mm   0
body_mass_g         0
sex                 0
year                0
dtype: int64
```

Question 1c) shows that only the columns

`['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']` contain missing values so we don't have to filter by those columns.

```
In [9]: penguins_clean.shape
```

```
Out[9]: (333, 8)
```

The cleaned data contains 333 rows and 8 columns.

Part 3) Numeric summary statistics

- a. Compute the following **for each numeric column**: count, mean, std, min, 25%, 50%, 75%, max (hint: `.describe()`)
- b. Compute the mean and std for each numeric column for each `species`

```
In [10]: penguins_clean.describe()
```

```
Out[10]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
count	333.000	333.000	333.000	333.000	333.000
mean	43.993	17.165	200.967	4207.057	2008.042
std	5.469	1.969	14.016	805.216	0.813
min	32.100	13.100	172.000	2700.000	2007.000
25%	39.500	15.600	190.000	3550.000	2007.000
50%	44.500	17.300	197.000	4050.000	2008.000
75%	48.600	18.700	213.000	4775.000	2009.000
max	59.600	21.500	231.000	6300.000	2009.000

```
In [11]: penguins_clean.drop(columns=["island", "sex"]).groupby("species").agg(['mean', 'std'])
```

Out[11]:

	bill_length_mm		bill_depth_mm		flipper_length_mm		body_mass_g		year	
	mean	std	mean	std	mean	std	mean	std	mean	std
species										
Adelie	38.824	2.663	18.347	1.219	190.103	6.522	3706.164	458.620	2008.055	0.812
Chinstrap	48.834	3.339	18.421	1.135	195.824	7.132	3733.088	384.335	2007.971	0.863
Gentoo	47.568	3.106	14.997	0.986	217.235	6.585	5092.437	501.476	2008.067	0.789

Part 4) Two-way grouped summaries

a) For each (**species**, **sex**) combination, compute the **mean** and **count** of:

- `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`.

b) Which (species, sex) has the **largest average body_mass_g**? Show the row.

In [12]: `penguins_clean.drop(columns=["island", "year"]).groupby(["species", "sex"]).agg(['mean', 'count'])`

Out[12]:

		bill_length_mm		bill_depth_mm		flipper_length_mm		body_mass_g	
		mean	count	mean	count	mean	count	mean	count
species	sex								
Adelie	female	37.258	73	17.622	73	187.795	73	3368.836	73
	male	40.390	73	19.073	73	192.411	73	4043.493	73
Chinstrap	female	46.574	34	17.588	34	191.735	34	3527.206	34
	male	51.094	34	19.253	34	199.912	34	3938.971	34
Gentoo	female	45.564	58	14.238	58	212.707	58	4679.741	58
	male	49.474	61	15.718	61	221.541	61	5484.836	61

In [13]: `penguins_clean.drop(columns=["island", "year"]).groupby(["species", "sex"]).mean().iloc[5,:]`

Out[13]: `bill_length_mm 49.474`
`bill_depth_mm 15.718`
`flipper_length_mm 221.541`
`body_mass_g 5484.836`
`Name: (Gentoo, male), dtype: float64`

The male Gentoo penguins have the largest average `body_mass_g`.

Part 5) Correlations

- a. Compute the **correlation matrix** among numeric columns.
- b. Compute the correlation matrix **within each species** (hint: groupby).
- c. Find an example of Simpson's Paradox in the data.

https://en.wikipedia.org/wiki/Simpson%27s_paradox

- An example Simpson's Paradox would be a pair of variables that have a positive correlation within subgroups, but when all the groups are combined, there is negative correlation (or vice-versa).

```
In [14]: penguins_clean.corr(numeric_only=True)
```

Out[14]:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
bill_length_mm	1.000	-0.229	0.653	0.589	0.033
bill_depth_mm	-0.229	1.000	-0.578	-0.472	-0.048
flipper_length_mm	0.653	-0.578	1.000	0.873	0.151
body_mass_g	0.589	-0.472	0.873	1.000	0.022
year	0.033	-0.048	0.151	0.022	1.000

```
In [15]: penguins_clean.groupby("species").corr(numeric_only=True)
```

Out[15]:

		bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
species						
Adelie	bill_length_mm	1.000	0.386	0.332	0.544	0.011
	bill_depth_mm	0.386	1.000	0.311	0.580	-0.235
	flipper_length_mm	0.332	0.311	1.000	0.465	0.328
	body_mass_g	0.544	0.580	0.465	1.000	-0.046
	year	0.011	-0.235	0.328	-0.046	1.000
Chinstrap	bill_length_mm	1.000	0.654	0.472	0.514	0.042
	bill_depth_mm	0.654	1.000	0.580	0.604	-0.059
	flipper_length_mm	0.472	0.580	1.000	0.642	0.346
	body_mass_g	0.514	0.604	0.642	1.000	0.037
	year	0.042	-0.059	0.346	0.037	1.000
Gentoo	bill_length_mm	1.000	0.654	0.664	0.667	0.205
	bill_depth_mm	0.654	1.000	0.711	0.723	0.243
	flipper_length_mm	0.664	0.711	1.000	0.711	0.201
	body_mass_g	0.667	0.723	0.711	1.000	0.052
	year	0.205	0.243	0.201	0.052	1.000

The variables `bill_depth_mm` and `bill_length_mm` have a negative correlation when grouped together, but when they are seperated by species they have a positive correlation.

Plots

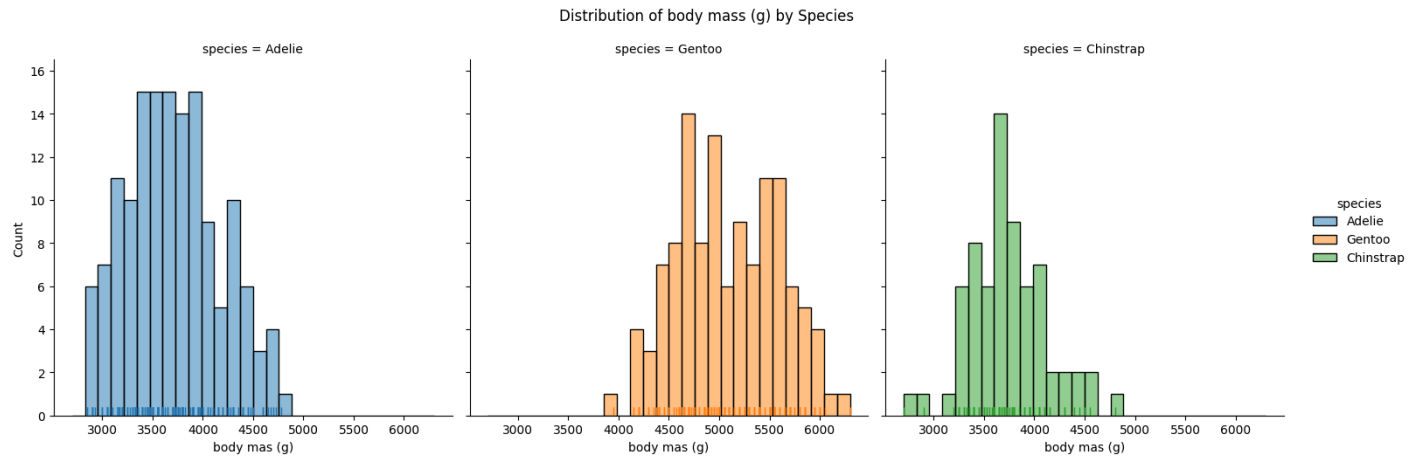
- Each figure must have a **title**, **axis labels**, and a **legend** (when appropriate).
- Use readable tick labels and sensible limits.
- Be sure each graphic is displayed.

Part 6) Histograms of body mass by species

Create **separate** histograms of `body_mass_g` for each species (three panels).

- Use the same binning across panels so comparisons are fair.
- Label axes clearly

```
In [16]: g = sns.displot(data=penguins_clean, x= "body_mass_g", hue="species", col="species", bins = 28,
g.figure.suptitle("Distribution of body mass (g) by Species", y = 1.05)
g.set_axis_labels("body mas (g)")
plt.show()
```



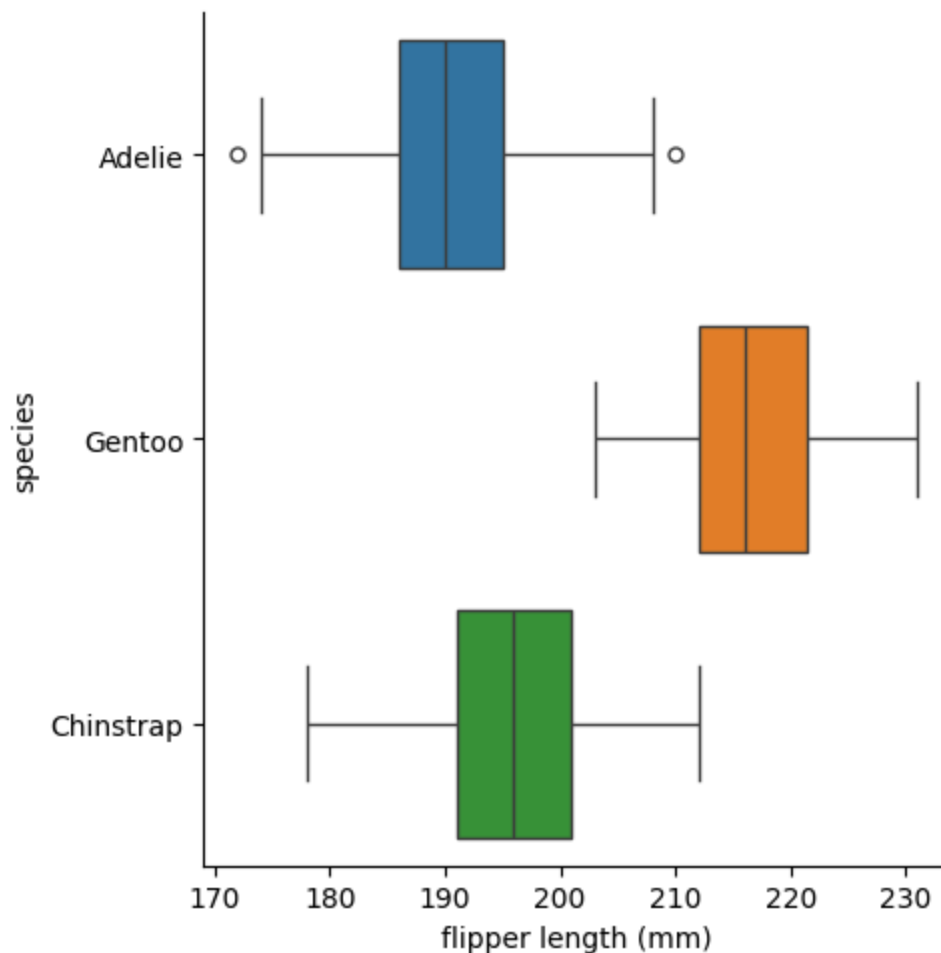
Part 7) Boxplots of flipper length

Make a **single figure** with boxplots of `flipper_length_mm` by **species**.

Briefly compare the medians and spreads in 2–3 sentences in a markdown cell.

```
In [17]: g = sns.catplot(data=penguins_clean, x= "flipper_length_mm", y="species", hue="species", kind="b
g.figure.suptitle("Boxplots of flipper length (mm) by Species", y = 1.05)
g.set_axis_labels("flipper length (mm)")
plt.show()
```

Boxplots of flipper length (mm) by Species



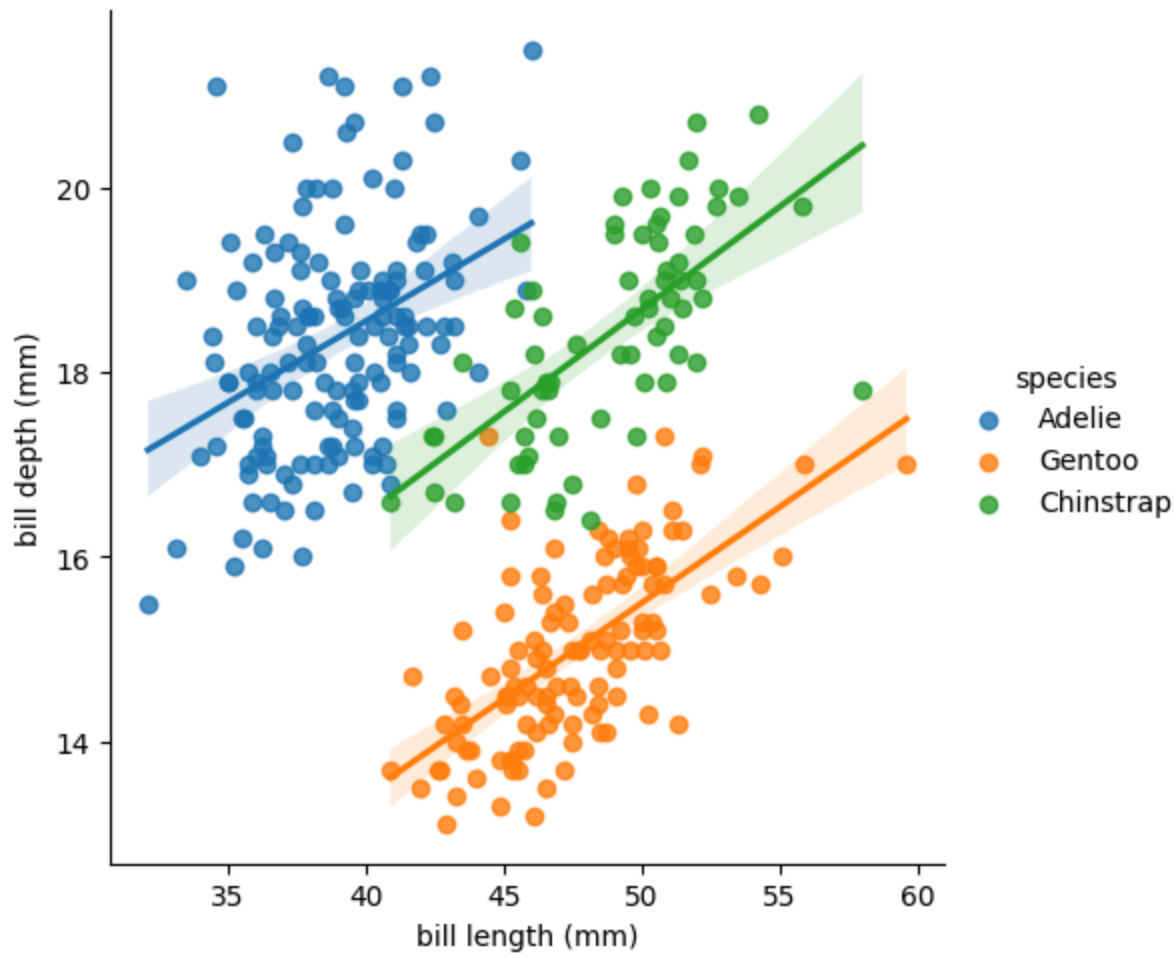
The Adelie penguins have the lowest mean (190), followed by the Chinstrap (~195), and Gentoo have the highest mean (~215). The Adelie penguins have the largest spread, and is the only one that contains outliers, while the Chinstrap penguins have the smallest spread.

Part 8) Scatter: bill length vs bill depth

- Make a scatter plot of **bill_length_mm** (x) vs **bill_depth_mm** (y), colored by species.
- Add a simple **linear fit line** per species
- Include a legend

```
In [18]: g = sns.lmplot(data=penguins_clean, x="bill_length_mm", y="bill_depth_mm", hue="species")
g.figure.suptitle("Scatterplot of bill length (mm) vs bill depth (mm) by Species", y = 1.05)
g.set_axis_labels("bill length (mm)", "bill depth (mm)")
plt.show()
```

Scatterplot of bill length (mm) vs bill depth (mm) by Species



Part 9) Stacked bars: sex proportions by species

Compute counts of `sex` within each `species`, convert to **proportions**, and make a **stacked bar chart**.

- Bars should be species on the x-axis with male/female proportions stacked to 1.0.

```
In [19]: g = pd.crosstab(penguins_clean.species, penguins_clean.sex, normalize=0).plot(kind="bar", stacked=True)
g.set_title('Stacked bar chart of proportions of sex within each species')
g.set_ylabel('Proportion')
plt.legend(bbox_to_anchor=(1.05, 1), loc = "upper left")
plt.show()
```


Stacked bar chart of proportions of sex within each species

